Cedrik Jean-Baptiste
March 2026
CIS 454

# Adapter Pattern Homework

**Chosen Adapter format: CSV**

**Test Cases:**

1. **TEST CASE 1**: Matching insertion and retrieval for a single employee
    a. Purpose: This test showcases that the CSV adapter stores and returns an employee in the same way as the JSON adapter

```python
# TEST CASE 1: Matching insert and retrieval for a single employee
# Purpose: This test showcases that the CSV adapter stores and returns an employee in the same way as the JSON adapter
def test_1():
    print("\n")
    print("TEST 1: Matching insert and retrieval for a single employee")
    json_file_1 = Json(FILE_PATH="test_1_employee_records_DB.json")
    csv_file_1 = Csv(FILE_PATH="test_1_employee_records_DB.csv")

    json_file_1.insert_employee("John", "Doe", "1980-01-15", "2020-06-10", "Software Engineer")
    csv_file_1.insert_employee("John", "Doe", "1980-01-15", "2020-06-10", "Software Engineer")
    json_result = json_file_1.get_employee("John", "Doe")
    csv_result = csv_file_1.get_employee("John", "Doe")

    print("JSON output: ", json_result)
    print("CSV  output: ", csv_result)
    print("json_output == csv_output: ", json_result == csv_result)
    print("\n")
```

2. **TEST CASE 2**: Matching “employee not found”
    a. Purpose: This test case showcases that both the CSV and JSON adapters return "None" when the requested employee does not exist

```python
# TEST CASE 2: Matching "employee not found"
# Purpose: This test case showcases that both the CSV and JSON adapters return "None" when the requested employee does not exist
def test_2():
    print("\n")
    print("TEST 2: Matching employee not found")
    json_file_2 = Json(FILE_PATH="test_2_employee_records_DB.json")
    csv_file_2 = Csv(FILE_PATH="test_2_employee_records_DB.csv")

    json_result = json_file_2.get_employee("Employee", "Not Found")
    csv_result = csv_file_2.get_employee("Employee", "Not Found")

    print("JSON output: ", json_result)
    print("CSV  output: ", csv_result)
    print("json_output == None and csv_output == None: ", ((json_result == None) and (csv_result == None)))
    print("\n")
```

3. **TEST CASE 3**: Matching get_all_employees data
    a. Purpose: This test showcases that full record retrieval (done through a call to the get_all_employees() method) for the CSV and JSON adapters returns the same keys and values

```python
# TEST CASE 3: Matching get_all_employees data
# Purpose: This test showcases that full record retrieval for the CSV and JSON adapters returns the same keys and values
def test_3():
    print("\n")
    print("TEST 3: Matching get_all_employees data")
    json_file_3 = Json(FILE_PATH="test_3_employee_records_DB.json")
    csv_file_3 = Csv(FILE_PATH="test_3_employee_records_DB.csv")

    json_file_3.insert_employee("John", "Doe", "1980-01-15", "2020-06-10", "Software Engineer")
    csv_file_3.insert_employee("John", "Doe", "1980-01-15", "2020-06-10", "Software Engineer")
    json_file_3.insert_employee("Jane", "Smith", "1990-05-20", "2021-03-15", "Project Manager")
    csv_file_3.insert_employee("Jane", "Smith", "1990-05-20", "2021-03-15", "Project Manager")
    json_result = json_file_3.get_all_employees()
    csv_result = csv_file_3.get_all_employees()

    all_match = True
    for i in range(len(json_result)):
        j = json_result[i]
        c = csv_result[i]
        if ((j["id"] != c["id"]) or (j["first_name"] != c["first_name"]) or (j["last_name"] != c["last_name"]) or (j["birthdate"] != c["birthdate"])):
            all_match = False
        elif ((j["hire_date"] != c["hire_date"]) or (j["job_title"] != c["job_title"])):
            all_match = False

    print("JSON output: ", json_result)
    print("CSV output: ", csv_result)
    print("len(json_output) == len(csv_output): ", len(json_result) == len(csv_result))
    print("All entries match: ", all_match)
    print("\n")
```

4. **TEST CASE 4**: Matching output format string
    a. Purpose: This test compares both the JSON and CSV adapter outputs against an expected string to ensure that the outputs are correct and matching

```python
# TEST CASE 4: Matching output format string
# Purpose: This test compares both the JSON and CSV adapter outputs against an expected string to ensure that the outputs are correct and matching
def test_4():
    print("\n")
    print("TEST 4: Matching output format string")
    json_file_4 = Json(FILE_PATH="test_4_employee_records_DB.json")
    csv_file_4 = Csv(FILE_PATH="test_4_employee_records_DB.csv")

    json_file_4.insert_employee("John", "Doe", "1980-01-15", "2020-06-10", "Software Engineer")
    csv_file_4.insert_employee("John", "Doe", "1980-01-15", "2020-06-10", "Software Engineer")
    json_result = json_file_4.get_employee("John", "Doe")
    csv_result = csv_file_4.get_employee("John", "Doe")

    expected = "ID: 1, Name: John Doe, Birthdate: 1980-01-15, Hire Date: 2020-06-10, Job Title: Software Engineer"

    print("Expected output: ", expected)
    print("JSON output: ", json_result)
    print("CSV output: ", csv_result)
    print("json_output == expected_output: ", json_result == expected)
    print("csv_output == expected_output: ", csv_result == expected)
    print("json_output == csv_output: ", json_result == csv_result)
    print("\n")
```

**Test Outputs:**

1. **TEST CASE 1**:
    a. Terminal Output:

```
TEST 1: Matching insert and retrieval for a single employee
JSON output:  ID: 1, Name: John Doe, Birthdate: 1980-01-15, Hire Date: 2020-06-10, Job Title: Software Engineer
CSV  output:  ID: 1, Name: John Doe, Birthdate: 1980-01-15, Hire Date: 2020-06-10, Job Title: Software Engineer
json_output == csv_output:  True
```

    b. JSON file:

```
{} test_1_employee_records_DB.json > ...
1  [
2      {
3          "id": 1,
4          "first_name": "John",
5          "last_name": "Doe",
6          "birthdate": "1980-01-15",
7          "hire_date": "2020-06-10",
8          "job_title": "Software Engineer"
9      }
10 ]
```

c. CSV file:

| | A | B | C | D | E | F | G |
|---|---|---|---|---|---|---|---|
| 1 | id | first_name | last_name | birthdate | hire_date | job_title | |
| 2 | 1 | John | Doe | ######## | ######## | Software Engineer | |
| 3 | | | | | | | |

2. **TEST CASE 2**:
   a. Terminal Output:

```
TEST 2: Matching employee not found
JSON output:  None
CSV  output:  None
json_output == None and csv_output == None:  True
```

b. JSON file:

```
{} test_2_employee_records_DB.json
1  []
```

c. CSV file:

| | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| 1 | id | first_name | last_name | birthdate | hire_date | job_title |
| 2 | | | | | | |

3. **TEST CASE 3**:
   a. Terminal Output:

```
TEST 3: Matching get_all_employees data
JSON output:  [{'id': 1, 'first_name': 'John', 'last_name': 'Doe', 'birthdate': '1980-01-15', 'hire_date': '2020-06-10', 'job_title': 'Software Engineer'}, {'id': 2, 'first_name': 'Jane',
'last_name': 'Smith', 'birthdate': '1990-05-20', 'hire_date': '2021-03-15', 'job_title': 'Project Manager'}]
CSV output:  [{'id': 1, 'first_name': 'John', 'last_name': 'Doe', 'birthdate': '1980-01-15', 'hire_date': '2020-06-10', 'job_title': 'Software Engineer'}, {'id': 2, 'first_name': 'Jane',
'last_name': 'Smith', 'birthdate': '1990-05-20', 'hire_date': '2021-03-15', 'job_title': 'Project Manager'}]
len(json_output) == len(csv_output):  True
All entries match:  True
```

b. JSON file:

{} test_3_employee_records_DB.json > ...

```
[
    {
        "id": 1,
        "first_name": "John",
        "last_name": "Doe",
        "birthdate": "1980-01-15",
        "hire_date": "2020-06-10",
        "job_title": "Software Engineer"
    },
    {
        "id": 2,
        "first_name": "Jane",
        "last_name": "Smith",
        "birthdate": "1990-05-20",
        "hire_date": "2021-03-15",
        "job_title": "Project Manager"
    }
]
```

c. CSV file:

| | A | B | C | D | E | F | G |
|---|---|---|---|---|---|---|---|
| 1 | id | first_name | last_name | birthdate | hire_date | job_title | |
| 2 | 1 | John | Doe | ######## | ######## | Software Engineer | |
| 3 | 2 | Jane | Smith | ######## | ######## | Project Manager | |

4. **TEST CASE 4**:
   a. Terminal Output:

```
TEST 4: Matching output format string
Expected output:  ID: 1, Name: John Doe, Birthdate: 1980-01-15, Hire Date: 2020-06-10, Job Title: Software Engineer
JSON output:  ID: 1, Name: John Doe, Birthdate: 1980-01-15, Hire Date: 2020-06-10, Job Title: Software Engineer
CSV output:  ID: 1, Name: John Doe, Birthdate: 1980-01-15, Hire Date: 2020-06-10, Job Title: Software Engineer
json_output == expected_output:  True
csv_output == expected_output:  True
json_output == csv_output:  True
```

b. JSON file:

```
{} test_4_employee_records_DB.json > ...
1  [
2      {
3          "id": 1,
4          "first_name": "John",
5          "last_name": "Doe",
6          "birthdate": "1980-01-15",
7          "hire_date": "2020-06-10",
8          "job_title": "Software Engineer"
9      }
10 ]
```

c. CSV file:

| | A | B | C | D | E | F | G |
|---|---|---|---|---|---|---|---|
| 1 | id | first_name | last_name | birthdate | hire_date | job_title | |
| 2 | 1 | John | Doe | ######## | ######## | Software Engineer | |